

Name: Carlos Bracho
Professor: Jeff Sanford

CS 499 5-2 Milestone Four: Enhancement Three: Databases

Artifact Description:

The artifact selected for this enhancement continues to be my full-stack inventory application developed as part of my Computer Science Capstone project. This is an application that utilizes the MERN stack, consisting of MongoDB, Express, React, and Node.js. The purpose of the application is to allow users to manage inventory items by creating, viewing, updating, and deleting products. Each product includes attributes such as name, price, image, quantity, and bin location, which are stored and managed within a MongoDB database.

Justification for Inclusion:

I selected this artifact for my ePortfolio because it demonstrates my ability to design and implement databases to applications. This project highlights my understanding of database schema design, CRUD operations, and the integration between frontend and backend systems.

During this enhancement, I focused on improving the database functionality and overall data integrity of the application. One major improvement was modifying the product schema to enforce uniqueness constraints on key fields such as product name and bin location. This prevents duplicate entries. Additionally, I implemented full CRUD functionality connected to the MongoDB database. I developed logic to allow users to create new products through a form, display all products dynamically on the homepage, update existing products using a modal interface, and delete products. These operations required careful coordination between the frontend and backend.

To manage application state and improve database interactions, I integrated Zustand into the project. This allowed me to centralize the logic for creating, updating, and deleting products,

improving both code organization and maintainability. I also enhanced user feedback by adding toast notifications to confirm successful or failed operations.

The frontend was structured using React Router to create a multi-page experience without full page reloads, including a Home Page for displaying products and a Create Page for adding new items. The product display was created using a grid layout and reusable components, such as a ProductCard, which visually presents database data including quantity and bin location.

Course Outcomes:

This enhancement demonstrates progress toward multiple course outcomes. First, it reflects my ability to design and evaluate computing solutions by developing a full-stack inventory management system that integrates a MongoDB database with a React frontend and Express backend. I implemented and refined CRUD operations, enforced schema constraints such as unique fields, and managed trade-offs between usability and data integrity when designing the database structure.

Second, this project highlights my ability to use modern tools and techniques to implement computing solutions. I utilized technologies such as MongoDB Atlas for cloud-based data storage, Zustand for state management, and APIs to handle communication between the frontend and backend. Finally, I began incorporating aspects of a security mindset by enforcing data validation and uniqueness constraints at the database level. These measures help prevent inconsistent or duplicate data and lay the groundwork for future enhancements such as authentication and access control.

Reflection on the Enhancement Process:

Throughout this enhancement, I gained a deeper understanding of how frontend and backend systems interact with a database. One of the key lessons I learned was the importance of

maintaining data integrity, particularly through schema design and validation. Implementing unique constraints helped me understand how to prevent duplicate or inconsistent data at the database level.

I also faced several challenges during development, particularly with debugging issues related to state management and data flow between components. For example, ensuring that the product list updated correctly after creating, deleting, or modifying items required careful handling of operations and state updates. These challenges helped me improve my problem-solving skills and better understand how data flows through a full-stack application.

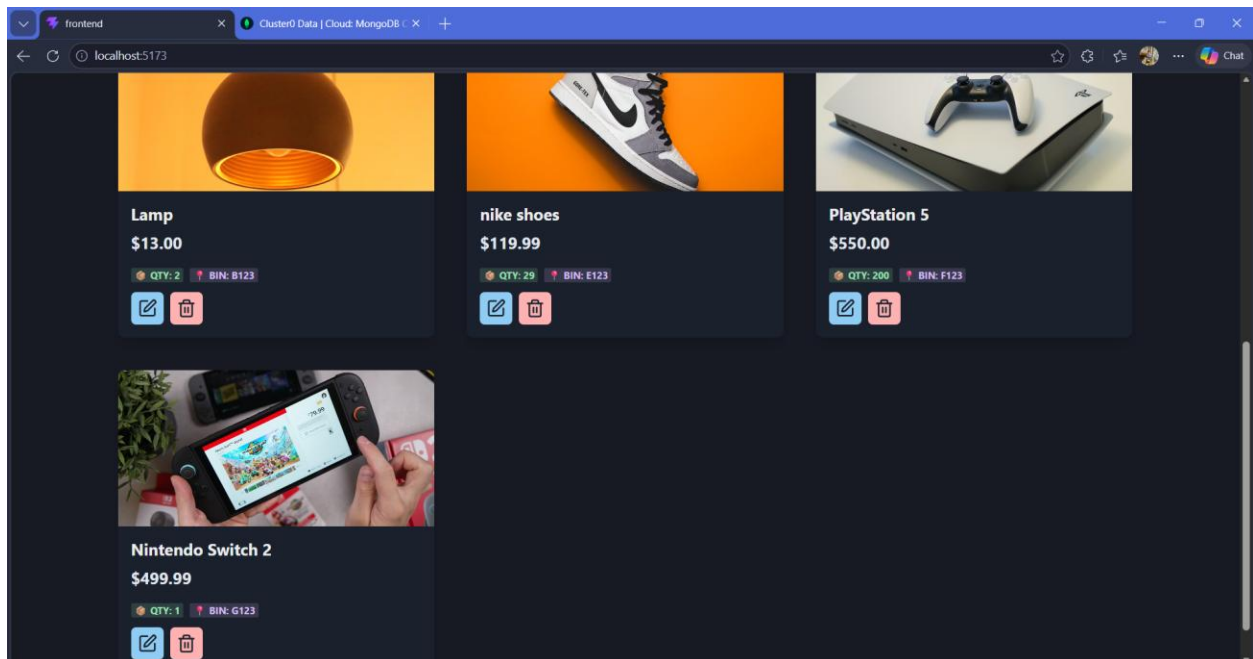
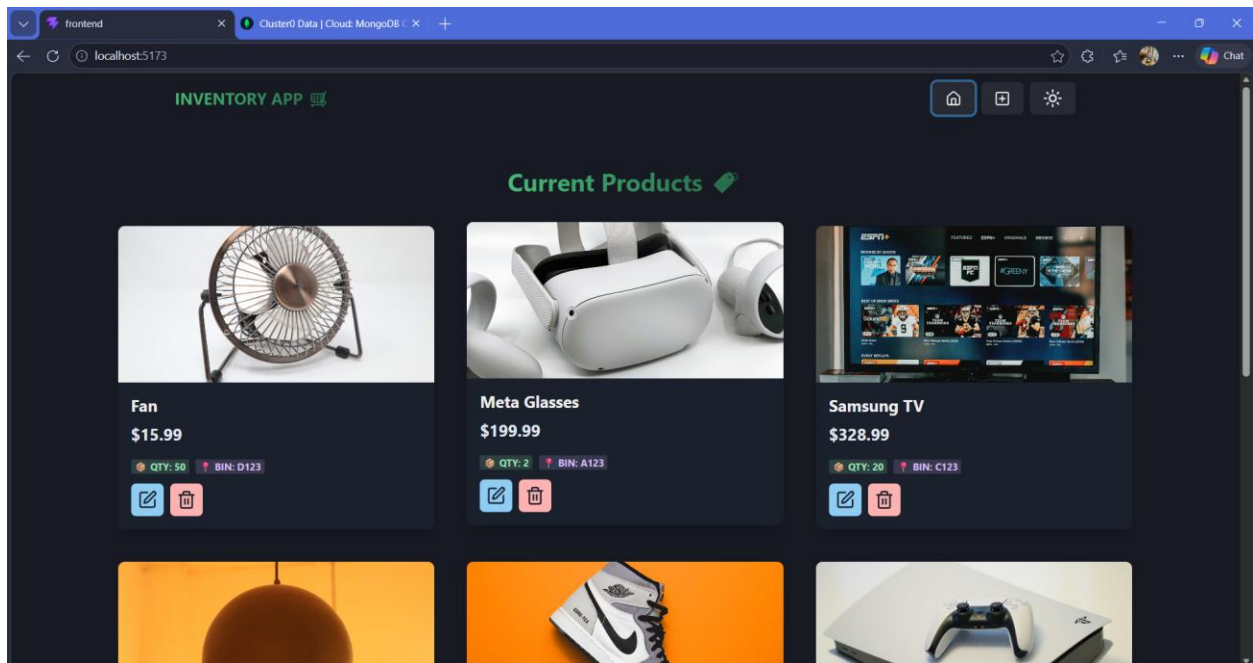
Another important takeaway was learning how to structure code in a more modular and maintainable way. By organizing logic into separate components and using a centralized store for database interactions, I was able to create a cleaner and more scalable codebase.

Overall, this enhancement strengthened my confidence in working with databases and reinforced my ability to build web applications. Moving forward, I plan to expand this project by implementing user authentication and deploying the application.

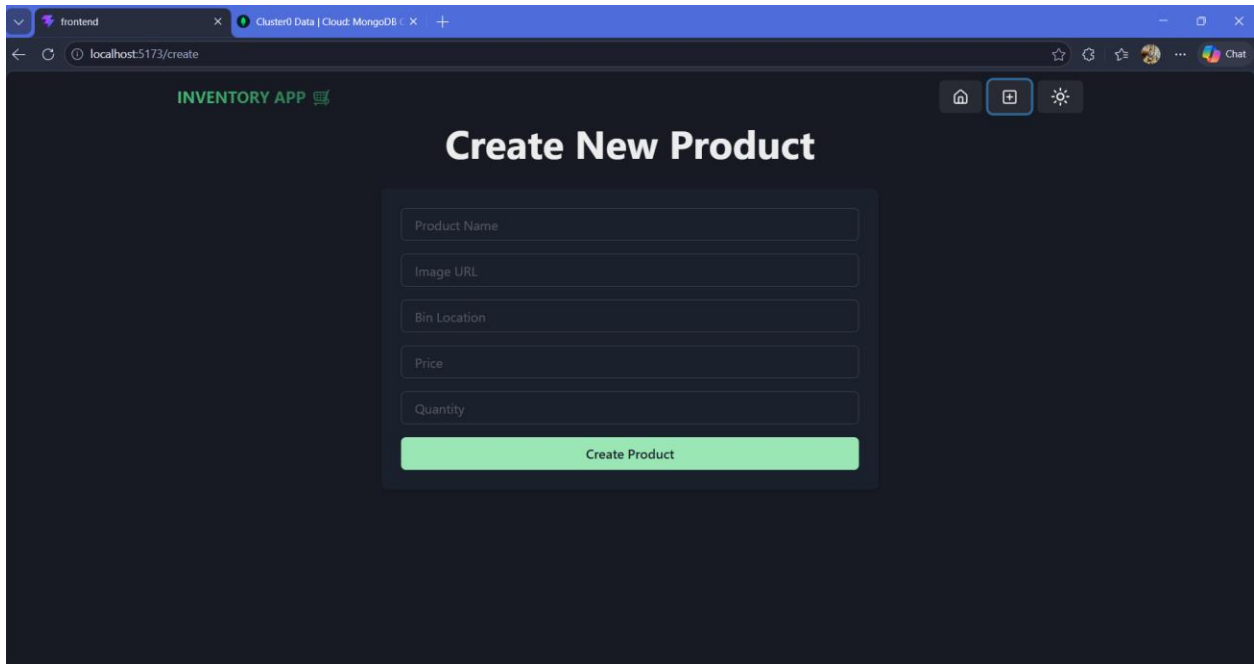
Visual Evidence of Application Functionality

To further demonstrate the functionality and usability of the application, the following images provide a visual representation of the user interface and its connection to the database:

- 1) **Home Page:** This page displays all current products stored in the database. Users can view product details and perform actions such as updating or deleting items directly from this interface.

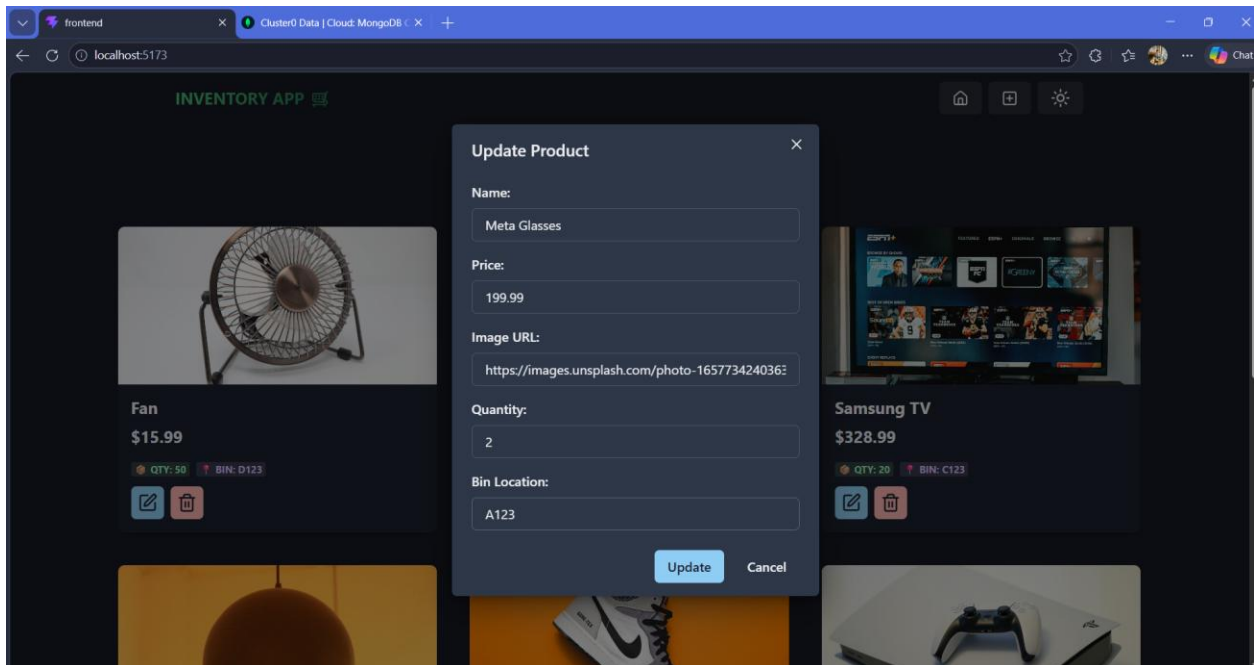


- 2) **Create Product Page:** This page allows users to input new product information, including name, price, quantity, image, and bin location. Upon submission, the data is stored in the MongoDB database and reflected on the home page.



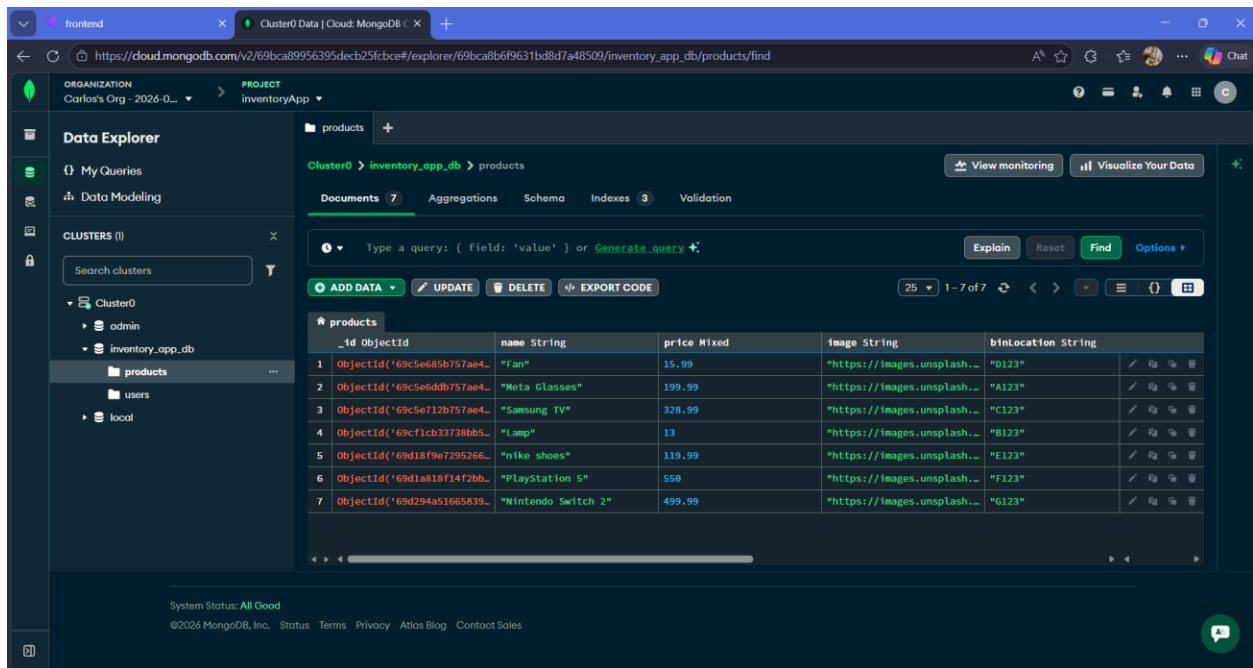
The screenshot shows a web browser window with the URL `localhost:5173/create`. The page title is "INVENTORY APP". The main heading is "Create New Product". Below the heading is a form with the following fields: "Product Name", "Image URL", "Bin Location", "Price", and "Quantity". At the bottom of the form is a green button labeled "Create Product".

- 3) **Update Product Modal:** This modal provides a way for users to update existing product information without navigating away from the home page. The form is pre-filled with the current product data, improving usability and efficiency.



The screenshot shows the "INVENTORY APP" home page with a modal titled "Update Product" open. The modal contains the following fields: "Name:" (pre-filled with "Meta Glasses"), "Price:" (pre-filled with "199.99"), "Image URL:" (pre-filled with "https://images.unsplash.com/photo-1657734240362"), "Quantity:" (pre-filled with "2"), and "Bin Location:" (pre-filled with "A123"). At the bottom of the modal are two buttons: "Update" and "Cancel". The background shows a grid of product cards, including a "Fan" for \$15.99 and a "Samsung TV" for \$328.99.

- 4) **MongoDB Atlas Product Collection:** This image shows the product collection stored in MongoDB Atlas. The data displayed on the home page directly corresponds to the records in the database, demonstrating successful integration between the frontend and backend systems.



- 5) **Empty Database State:** This image illustrates how the application handles cases where no products are available. The interface provides a clear message and guidance to the user, ensuring a smooth user experience even when the database is empty or unavailable.

